

Banking Application – Class Design

Introduction

This document presents the object-oriented class design for a **Banking Server Application**.

The goal of the design is to model real-world banking operations using **OOP principles** such as **encapsulation, abstraction, modularity, and responsibility separation**.

The application supports the following banking functionalities:

- Balance inquiry
- Deposit
- Withdrawal with balance validation
- Fund transfer between accounts
- Viewing the last 10 transactions
- Account validation and error handling

The design is represented using a **UML Class Diagram**, and the design process is divided into **three days**, as specified in the assignment.

Day 1

Problem Analysis & Entity Identification

Functional Requirements

The banking system should:

- Allow customers to check their account balance
- Support depositing money into an account
- Support withdrawing money without allowing negative balances
- Enable fund transfer between two valid accounts
- Keep a record of all transactions
- Allow viewing of the **last 10 transactions** for an account
- Validate account numbers before performing operations
- Handle invalid account and insufficient balance scenarios gracefully

Real-World Entities Involved

From the above requirements, the following real-world concepts emerge:

- A **Customer** who owns an account
 - A **Bank** that manages customers and accounts
 - An **Account** that stores balance and transactions
 - A **Transaction** representing every banking operation
 - A **Service layer** to coordinate operations
 - **Exceptions** to handle error scenarios
-

Identified Entities

Entity	Description
Customer	Represents a bank customer
Account	Represents a customer's bank account
Bank	Central authority managing accounts and customers
Transaction	Records deposit, withdrawal, and transfer operations
TransactionType	Enum defining transaction categories
AccountService	Service layer for banking operations
InvalidAccountException	Error for invalid account numbers
InsufficientBalanceException	Error for insufficient balance

Day 2

Attributes & Responsibilities

Entity-wise Attributes & Responsibilities

➤ Customer

Attributes

- customerId : String
- name : String
- email : String
- phone : String

Responsibilities / Methods

- getCustomerId() : String
- getName() : String
- getEmail() : String
- getPhone() : String

Explanation

- The Customer entity represents a bank customer.
- Attributes store essential identification and contact information required by the bank.
- Getter methods provide controlled access to customer data.
- No business logic is placed in Customer to keep it a simple data holder.
- This supports encapsulation by preventing direct access to internal fields.

➤ Account

Attributes

- accountNumber : String
- balance : double
- customer : Customer
- transactions : List<Transaction>

Responsibilities / Methods

- getAccountNumber() : String
- getBalance() : double
- deposit(amount : double) : void
- withdraw(amount : double) : boolean
- transfer(toAccount : Account, amount : double) : boolean
- addTransaction(tx : Transaction) : void
- getLast10Transactions() : List<Transaction>

Explanation

- The Account entity models a bank account and holds financial data.
- The balance attribute stores the current available amount.
- Deposit, withdrawal, and transfer methods perform core banking operations.
- Withdrawal and transfer methods enforce business rules such as non-negative balance.
- Transaction history is maintained to support auditing and account statements.
- The method to fetch the last 10 transactions improves performance and usability.

➤ Transaction

Attributes

- transactionId : String
- date : LocalDateTime
- amount : double
- type : TransactionType
- fromAccount : String
- toAccount : String

Responsibilities / Methods

- getTransactionId() : String
- getDate() : LocalDateTime
- getAmount() : double
- getType() : TransactionType

Explanation

- The Transaction entity represents a single financial operation.
- Attributes store all details required to uniquely identify and track a transaction.
- From and to account numbers help identify transfer direction.
- Getter methods allow read-only access to transaction data.
- Transactions are immutable to preserve data integrity and audit accuracy.

➤ TransactionType (Enum)

Values

- DEPOSIT
- WITHDRAWAL
- TRANSFER

Explanation

- The enum defines a fixed set of valid transaction types.
- It prevents invalid or inconsistent transaction type values.
- Using an enum improves readability and type safety.
- It simplifies transaction classification logic across the system.

➤ Bank

Attributes

- bankId : String
- bankName : String
- branchName : String
- ifscCode : String
- address : String
- accounts : Map<String, Account>

Responsibilities / Methods

- loadAccounts() : void
- getAccount(accountNumber : String) : Account
- validateAccount(accountNumber : String) : Boolean

Explanation

- The Bank entity acts as the central controller of the system.
- It stores and manages all accounts.
- Account validation ensures operations are performed only on valid accounts.
- Preloading accounts supports the requirement that accounts exist at startup.

➤ AccountService

Attributes

- bank : Bank

Responsibilities / Methods

- deposit(accountNumber : String, amount : double) : void
- withdraw(accountNumber : String, amount : double) : void
- transfer(fromAcc : String, toAcc : String, amount : double) : void
- checkBalance(accountNumber : String) : double

Explanation

- AccountService provides a simplified interface for clients.
- It delegates all operations to the Bank entity.
- This prevents direct access to domain objects like Account.
- Acts as a service boundary, supporting abstraction and clean architecture.
- Makes the system easier to extend and maintain.

➤ Exceptions

Exception Classes

- InvalidAccountException
- InsufficientBalanceException

Explanation

- InvalidAccountException handles cases where an account number does not exist.
- InsufficientBalanceException handles attempts to withdraw or transfer more than the available balance.
- Explicit exceptions make error handling clear and meaningful.
- They improve system robustness and readability.

Day 3

UML Class Diagram

PlantUML Code

```
@startuml
skinparam classAttributeIconSize 0
skinparam classFontStyle bold
skinparam shadowing true
skinparam linetype ortho
```

```
' =====
'      Customer
' =====
class Customer {
    - customerId : String
    - name : String
    - email : String
    - phone : String
    --
    + getCustomerId() : String
    + getName() : String
    + getEmail() : String
    + getPhone() : String
}
```

```
' =====
'      Account
' =====
class Account {
    - accountNumber : String
    - balance : double
```

```

- customer : Customer
- transactions : List<Transaction>
--
+ getAccountNumber() : String
+ getBalance() : double
+ deposit(amount : double) : void
+ withdraw(amount : double) : boolean
+ transfer(toAccount : Account, amount : double) : boolean
+ addTransaction(tx : Transaction) : void
+ getLast10Transactions() : List<Transaction>
}

```

```

' =====
'   Transaction
' =====
class Transaction {
    - transactionId : String
    - date : LocalDateTime
    - amount : double
    - type : TransactionType
    - fromAccount : String
    - toAccount : String
    --
    + getTransactionId() : String
    + getDate() : LocalDateTime
    + getAmount() : double
    + getType() : TransactionType
}

```

```

' =====
'   TransactionType Enum
' =====
enum TransactionType {
    DEPOSIT
    WITHDRAWAL
}

```

```
TRANSFER
}
```

```
' =====
'      Bank
' =====

class Bank {
    - bankId : String
    - bankName : String
    - branchName : String
    - ifscCode : String
    - address : String
    - accounts : Map<String, Account>
    --
    + loadAccounts() : void
    + getAccount(accountNumber : String) : Account
    + validateAccount(accountNumber : String) : boolean
}
```

```
' =====
'      AccountService
' =====

class AccountService {
    - bank : Bank
    --
    + deposit(accountNumber : String, amount : double) : void
    + withdraw(accountNumber : String, amount : double) : void
    + transfer(fromAcc : String, toAcc : String, amount : double) : void
    + checkBalance(accountNumber : String) : double
}
```

```
' =====
'      Exceptions
' =====
```

```
class InvalidAccountException
class InsufficientBalanceException
```

```
' =====
'  RELATIONSHIPS
' =====
```

```
' ☒ COMPOSITION: Customer is OWNED by Account
Customer "1" *-- "1" Account : owned by
```

```
' ☒ Aggregation: Bank has many accounts (accounts live independently)
Bank "1" o-- "10..*" Account : manages
```

```
' ☒ COMPOSITION: Account contains Transactions
Account "1" *-- "0..*" Transaction : records
```

```
' ☒ Enum relation
Transaction --> TransactionType : type
```

```
' ☒ Exceptions thrown by Account
Account --> InvalidAccountException : throws
Account --> InsufficientBalanceException : throws
```

```
' ☒ AccountService uses Bank
AccountService --> Bank : uses
```

```
' ☒ AccountService operates on Account
AccountService --> Account : operates
```

'  AccountService creates Transactions
AccountService --> Transaction : creates
@enduml

UML Class Diagram

